mastra x elastic x PLUGANDPLAY

Hybrid Retrieval Patterns for AI Agents

Mastra <> Elastic

Dan Goosewin, @goosewin
Developer Relations, Mastra
Sunnyvale · June 23, 2026

Data retrieval is hard.

Retrieval is where agents live or die in production. There are many ways to implement retrieval, and it’s hard to choose the correct approach.

- 5 implementation patterns
- ElasticSearch retrievers + Mastra agent in 60 lines of code
- Demo

Pure vector RAG is a great demo and a mediocre product.

An agent overwrites the user query before it ever searches. Paraphrase is great for meaning — and it quietly deletes the exact tokens vector search is worst at: error codes, SKUs, version numbers, proper nouns.

WHAT THE USER TYPED

"Lumio Hub v2 stuck in a reset loop after the 4.1 firmware"

Exact tokens: **Lumio Hub v2** · **4.1**

WHAT THE EMBEDDER SAW

"smart-home hub repeatedly restarting after a software update"

Lost: **the version string that names the exact bug.**

We have a collection of sci-fi movies.

Let’s figure out together how we can implement natural language search over it.

github.com/goosewin/mastra-elastic-demo

Five patterns.

01

Hybrid lexical + semantic

RRF fuses BM25 and vectors — no score tuning.

02

Two-stage reranking

Over-fetch cheap, reorder with a cross-encoder.

Enter mastra

03

Agentic retrieval

Retrieval becomes a tool model decides to call.

04

Agent-driven filtering

Natural language → structured metadata filters.

05

Memory as retrieval

The agent's history, recalled with same query.

01 Fuse lexical + semantic with RRF

BM25 wins exact tokens. **Vectors** win intent. **RRF** merges both by rank — score(d) = Σ 1/(k + rank$_i$), k≈60. No calibration. (retrievers GA in 8.16.)

Kibana · Dev Tools

```
POST scifi-movies/_search
{
  "retriever" : {
    "rrf" : {
      "retrievers" : [
        { "standard" : {
            "query" : { "match": { "metadata.description": "ufo first
contact" } } } },
        { "knn" : {
            "field" : "embedding",
            "query_vector" : [0.07, -0.21, 0.13],
            "k" : 50, "num_candidates" : 100 } }
      ],
      "rank_constant" : 60,
      "rank_window_size" : 100
    }
  }
}
```

PROS

Score-agnostic. BM25 is unbounded; cosine is [0,2]. RRF never has to reconcile them.

Plug-and-play. No per-dataset weight tuning like convex combination needs.

Sparse. Swap the knn leg for a sparse_vector / ELSER query — same rrf retriever.

02 Two-stage re-ranking

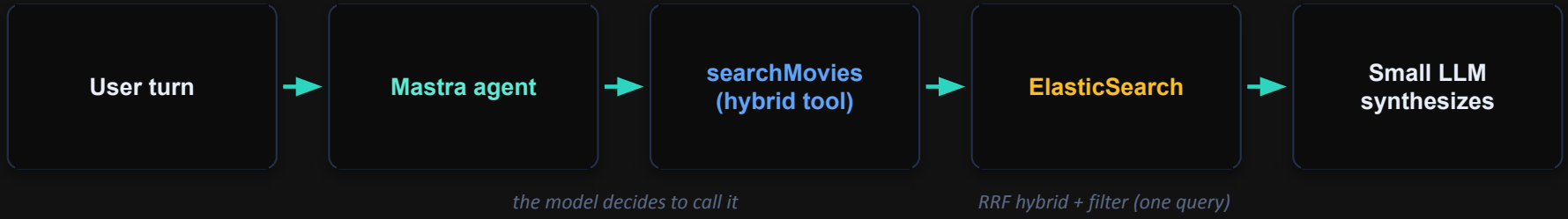
Over-fetch cheap, then rerank. BM25 and bi-encoders score query and doc **independently**. A cross-encoder scores them **jointly** — stronger signal, higher cost. So fetch wide, rerank a small pool.

Kibana · Dev Tools

```
POST scifi-movies/_search
{
  "retriever": {
    "text_similarity_reranker": {
      "retriever": {
        "rrf": { "retrievers": [
          { "standard": { "query": { "match": { "metadata.description": "ufo first contact" } } }
},
          { "knn": { "field": "embedding", "query_vector": [0.07, -0.21],
                     "k": 80, "num_candidates": 200 } }
        ], "rank_window_size": 100 }
      },
      "field": "metadata.description",
      "inference_id": ".jina-reranker-v3",
      "inference_text": "first contact with UFOs",
      "rank_window_size": 100
    }
  }
}
```

Enter mastra

A TypeScript AI agent framework. The part that matters here: retrieval is a **tool**, the model calls; and a model issues a single Elasticsearch **RRF** query (BM25 + vector).



Agent

decides, calls tools, loops over a turn

Tools

retrieval, APIs — the model chooses when

Memory

per-conversation context, one constructor

03 Retrieval as a tool the model calls

src/mastra/agents/elasticsearch-agent.ts

```
// src/mastra/tools/hybrid-search-tool.ts

export const searchMovies = createTool({
 id: "searchMovies",
 inputSchema: z.object({ query: z.string(), minYear: z.number().optional(),
topK: z.number().default(5) }),
 execute: async ({ query, topK }) => {
   const { embeddings } = await embeddingModel.doEmbed({ values: [query] });
   const res = await es.search({
     index: INDEX, size: topK,
     retriever: { rrf: { retrievers: [
       { standard: { query: { match: { "metadata.description": query } } } },
// BM25
       { knn: { field: "embedding", query_vector: embeddings[0], k: 50,
num_candidates: 100 } }, // vector
     ], rank_constant: 60 } },
   });
   return { results: res.hits.hits.map(h => h._source.metadata) };
 },
});
```

TOP-DOWN BREAKDOWN

store → where vectors live

tool → wraps the store as something callable

agent → owns the tool + memory and runs the loop

Not a pipeline. There's no hardcoded 'embed → search → stuff prompt.' The model calls the tool when it decides it needs context.

04 Let the agent write the filter

Half of real queries carry a structured constraint — a year, a status, a tenant. The agent translates language into a metadata filter that rides alongside the vector search.

tool.ts

```
// the model passes a year range; the tool turns it into an ES range filter

inputSchema: z.object({ query: z.string(), minYear: z.number().optional(), maxYear: z.number().optional() })

// → applied to BOTH legs:  { range: { "metadata.year": { gte: minYear } } }
```

This is where your dense_vector field meets a Boolean filter — the thing a bolt-on vector DB makes you orchestrate by hand.

USER

"UFO movies released after 2010"

AGENT QUERY PLAN

```
semantic: "UFO movies"
minYear: 2010 → range
metadata.year ≥ 2010
```

Structured + unstructured, one round-trip.

05 Memory

Memory is retrieval over the agent's own history. A follow-up ("which of those is the scariest?") reasons over the previous result set without re-searching. Backed by Mastra Memory (SQLite here; swap to any store).

Episodic

Time-stamped events — every turn as it lands. Decays.

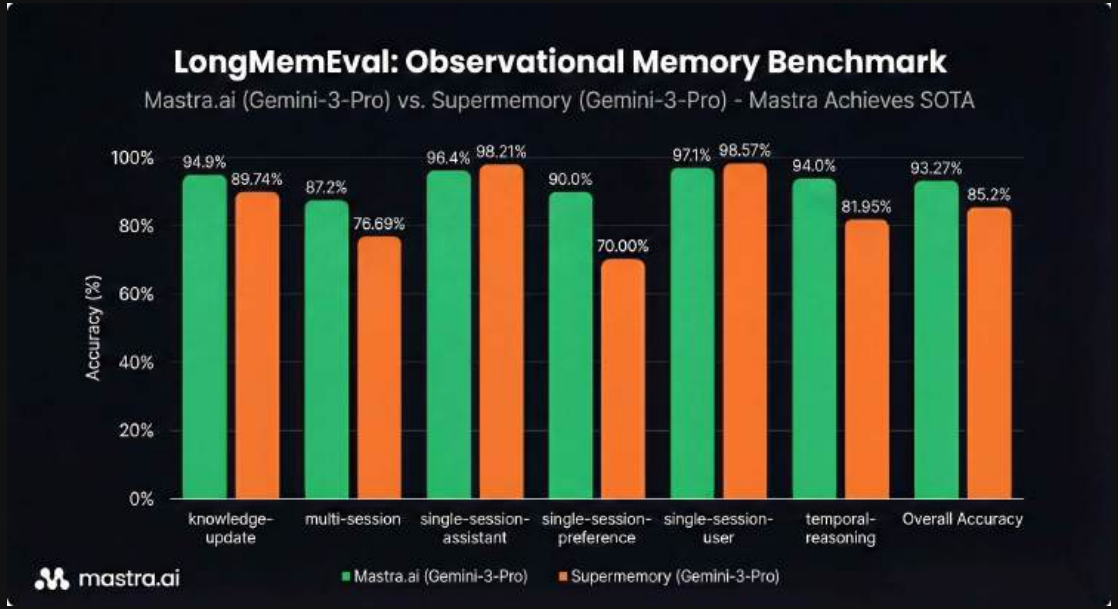
Semantic

Stable facts about the user. Curated, superseded.

Procedural

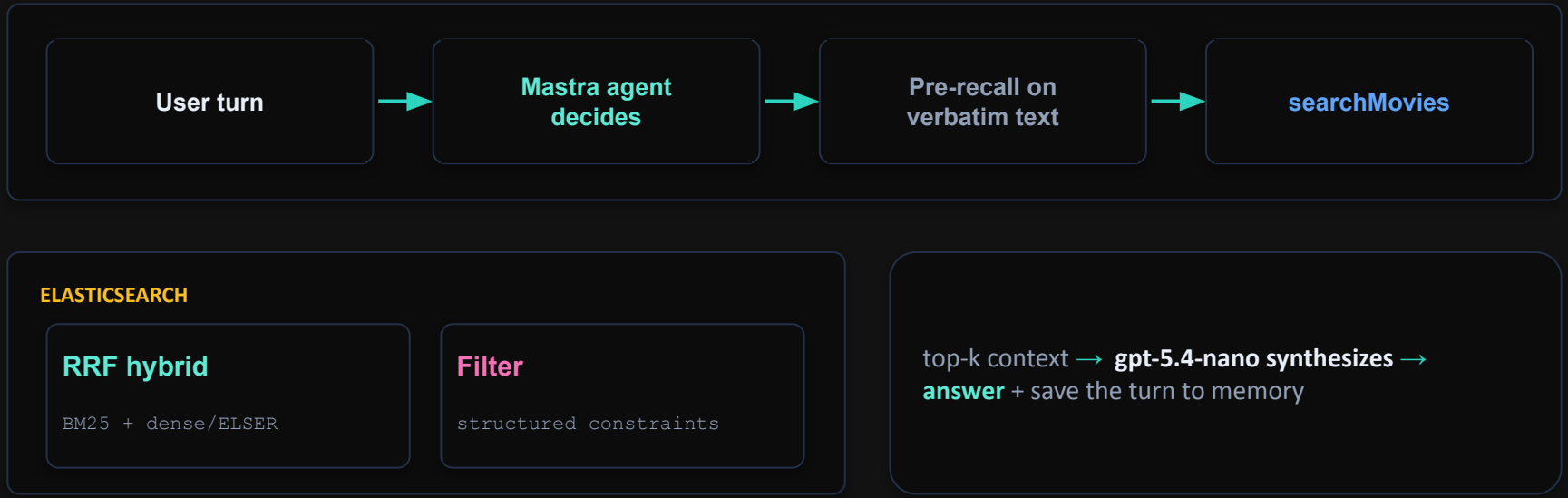
Playbooks. Carry success / failure counts.

Check out Observational Memory



mastra.ai/docs/memory/observational-memory

Five patterns, one query path



Thank you.

2026

mastra.ai

Mastra x Elastic · Sunnyvale · June 23, 2026